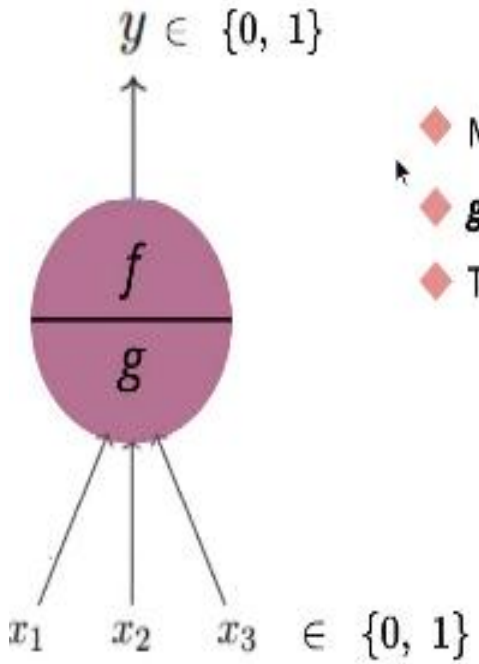
Unit 2
Deep Neural Networks

M-P Neuron Model



- McCulloch and Pitts proposed a highly simplified computational model of the neuron.
- ***g*** aggregates the inputs and the function ***f*** takes a decision based on this aggregation.
- The inputs can be excitatory or inhibitory

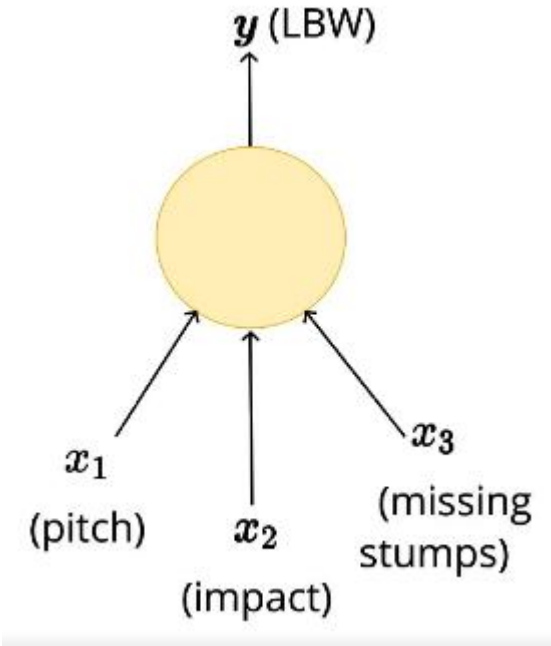
$y = 0$ if any x_i is inhibitory, else

$$g(x_1, x_2, ...x_n) = g(x) = \sum_{i=1}^{n} x_i$$

$$y = f(g(x)) = 1 \text{ if } g(x) \geq b$$

$$= 0 \text{ if } g(x) < b$$

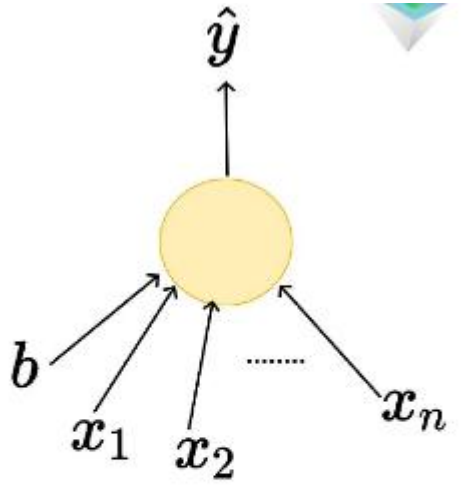
M-P Neuron – Data & Task





M-P Neuron – Loss Function

Launch (within 6 months)	0	1	1	0	0	1	0	1	1	0
Weight (<160g)	1	0	1	0	0	0	1	0	0	1
Screen size (<5.9 in)	1	0	1	0	1	0	1	0	1	0
dual sim	1	1	0	0	0	1	0	1	0	0
Internal memory (>= 64 GB, 4GB RAM)	1	1	1	1	1	1	1	1	1	0
NFC	0	1	1	0	1	0	1	1	1	0
Radio	1	0	0	1	1	1	0	0	0	0
Battery(>3500mAh)	0	0	0	1	0	1	0	1	0	0
Price > 20k	0	1	1	0	0	0	1	1	1	0
Like? (y)	1	0	**1**	**0**	1	1	**0**	**1**	0	0
prediction $\hat{y}$	1	0	**0**	**1**	1	1	**1**	**0**	0	0
loss	0	0	1	-1	0	0	-1	1	0	0



$$\hat{y} = \sum_{i=1}^{n} x_i \geq b$$

$$loss = \sum_i (y_i - \hat{y}_i)^2$$

M-P Neuron – Learning Algorithm

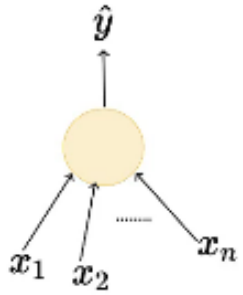
Launch (within 6 months)	0	1	1	0	0	1	0	1	1	0
Weight (<160g)	1	0	1	0	0	0	1	0	0	1
Screen size (<5.9 in)	1	0	1	0	1	0	1	0	1	0
dual sim	1	1	0	0	0	1	0	1	0	0
Internal memory (>= 64 GB, 4GB RAM)	1	1	1	1	1	1	1	1	1	0
NFC	0	1	1	0	1	0	1	1	1	0
Radio	1	0	0	1	1	1	0	0	0	0
Battery(>3500mAh)	0	0	0	1	0	1	0	1	0	0
Price > 20k	0	1	1	0	0	0	1	1	1	0
Like? (y)	1	1	1	0	0	1	1	1	0	0
prediction	?	?	?	?	?	?	?	?	?	?

Only one parameter, can afford Brute Force search

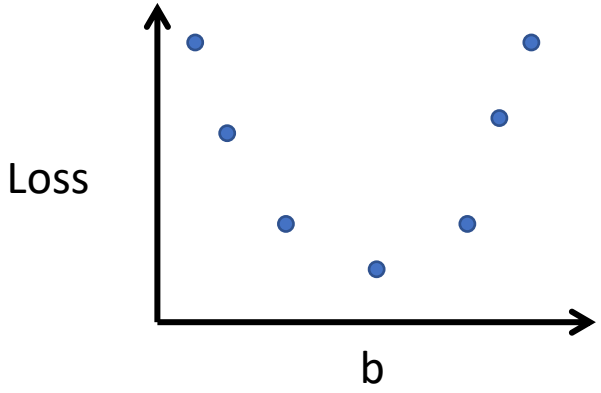
$$\hat{y} = \sum_{i=1}^{n} x_i \geq b$$

$$loss = \sum_i (y_i - \hat{y}_i)^2$$

$$b = 1?$$



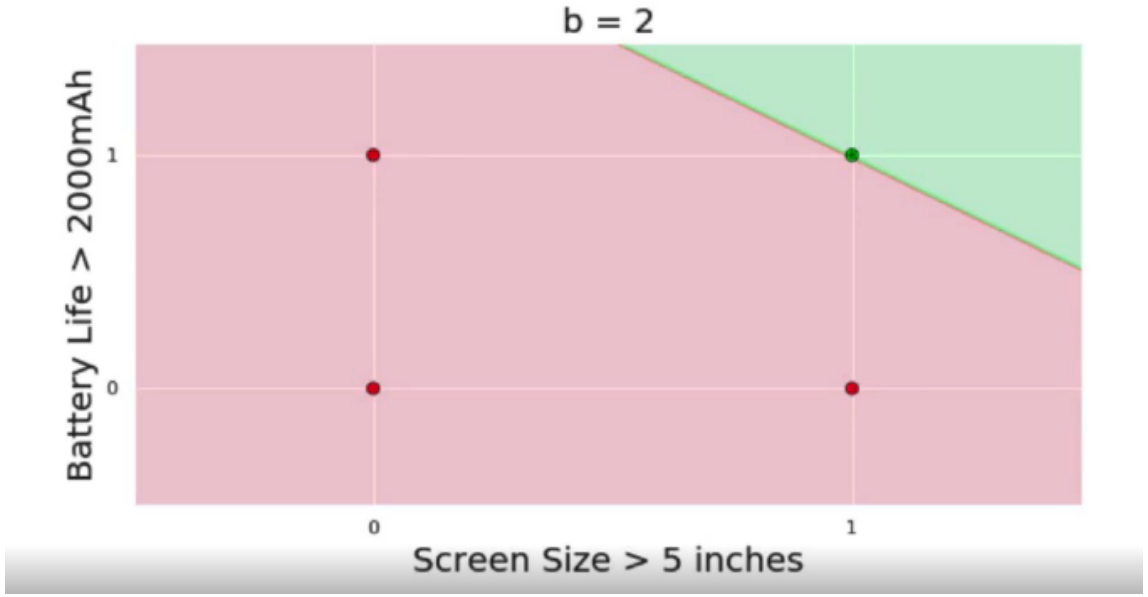
b varies from 0 to n

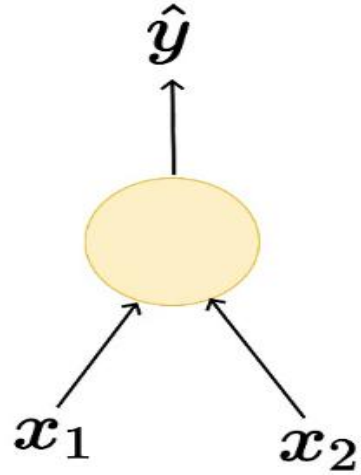


M-P Neuron – Geometric Interpretation



Screen size (>5 in)	1	0	1	0	1	0	1	0	1	0
Battery (>2000 mAh)	0	0	0	1	0	1	0	1	0	0
Like	1	0	1	0	1	1	0	1	0	0





$$\hat{y} = \left(\sum_{i=1}^{n} x_i \geq b\right)$$

- Boolean inputs
- Boolean output
- Linear
- Fixed Slope
- Few possible intercepts (b's)

Perceptron – Data & Task
Data Preparation

									
Launch (within 6 months)	0	1	1	0	0	1	0	1	1
Weight (g)	151	180	160	205	162	182	138	185	170
Screen size (inches)	5.8	6.18	5.84	6.2	5.9	6.26	4.7	6.41	5.5
dual sim	1	1	0	0	0	1	0	1	0
Internal memory (>= 64 GB, 4GB RAM)	1	1	1	1	1	1	1	1	1
NFC	0	1	1	0	1	0	1	1	1
Radio	1	0	0	1	1	1	0	0	0
Battery(mAh)	3060	3500	3060	5000	3000	4000	1960	3700	3260
Price (INR)	15k	32k	25k	18k	14k	12k	35k	42k	44k
Like (*y*)	**1**	**0**	**1**	**0**	**1**	**1**	**0**	**1**	**0**

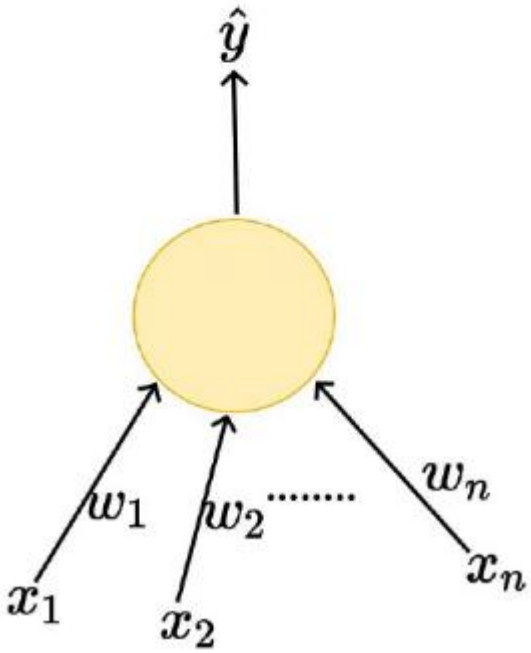
Standardization

$$x' = \frac{x - min}{max - min}$$

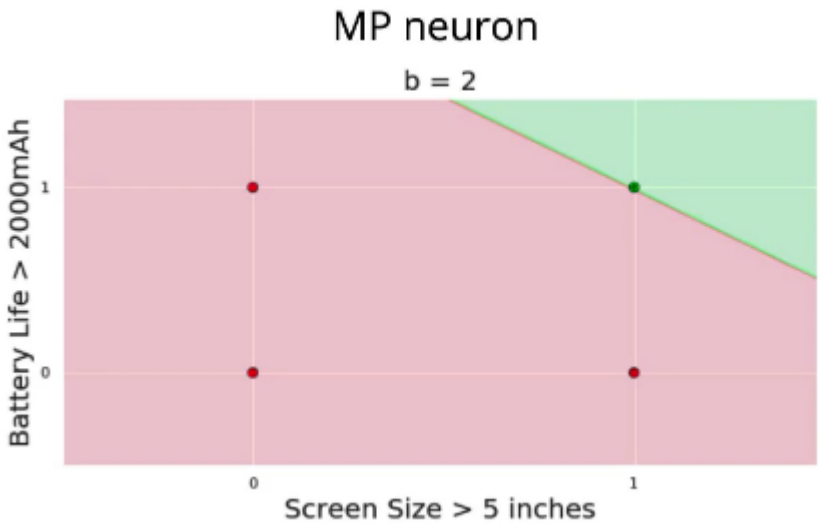
Perceptron – Model

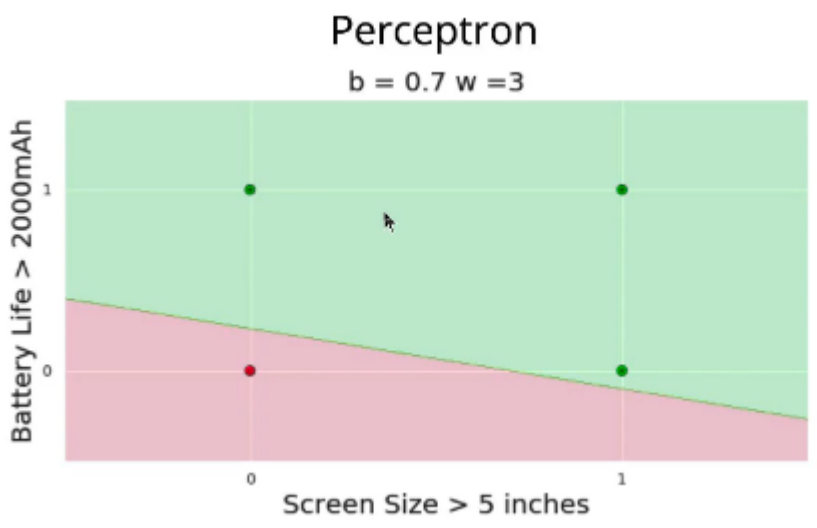
									
Launch (within 6 months)	0	1	1	0	0	1	0	1	1
Weight (g)	151	180	160	205	162	182	138	185	170
Screen size (inches)	5.8	6.18	5.84	6.2	5.9	6.26	4.7	6.41	5.5
dual sim	1	1	0	0	0	1	0	1	0
Internal memory (>= 64 GB, 4GB RAM)	1	1	1	1	1	1	1	1	1
NFC	0	1	1	0	1	0	1	1	1
Radio	1	0	0	1	1	1	0	0	0
Battery(mAh)	3060	3500	3060	5000	3000	4000	1960	3700	3260
Price (INR)	15k	32k	25k	18k	14k	12k	35k	42k	44k
Like (y)	**1**	**0**	**1**	**0**	**1**	**1**	**0**	**1**	**0**

$$\hat{y} = 1 \text{ if } \sum_{i=1}^{n} w_i x_i \geq b$$
$$\hat{y} = 0 \text{ otherwise}$$



Perceptron – Geometric Interpretation





Works well for linearly separable data

Perceptron – Learning Algorithm

Algorithm: Perceptron Learning Algorithm

$P \leftarrow$ *inputs with label* 1;

$N \leftarrow$ *inputs with label* 0;

Initialize **w** randomly;

while !*convergence* **do**

  Pick random $\mathbf{x} \in P \cup N$;

  if $\mathbf{x} \in P$ *and* $\sum_{i=0}^{n} w_i * x_i < 0$ **then**

    $\mathbf{w} = \mathbf{w} + \mathbf{x}$;

  end

  if $\mathbf{x} \in N$ *and* $\sum_{i=0}^{n} w_i * x_i \geq 0$ **then**

    $\mathbf{w} = \mathbf{w} - \mathbf{x}$;

  end

end

//the algorithm converges when all the inputs are classified correctly

$$\hat{y} = 1 \text{ (if } \sum_{i=0}^{n} w_i x_i \geq 0)$$
$$\hat{y} = 0 \text{ (otherwise)}$$

$$\hat{y} = 1 \text{ (if } \mathbf{w}.\mathbf{x} \geq 0)$$
$$\hat{y} = 0 \text{ (otherwise)}$$

For $\mathbf{x} \in P$ if $\mathbf{w}.\mathbf{x} < 0$ then it means that the angle (α) between this $\mathbf{x}$ and the current $\mathbf{w}$ is greater than 90° (but we want α to be less than 90°)

What happens to the new angle (α_{new}) when $\mathbf{w_{new}} = \mathbf{w} + \mathbf{x}$

$$\mathbf{w} \cdot \mathbf{x} = \|\mathbf{w}\| \, \|\mathbf{x}\| \cos\theta$$

$$\cos\theta = \frac{\mathbf{w} \cdot \mathbf{x}}{\|\mathbf{w}\| \, \|\mathbf{x}\|}$$

$$\begin{aligned} cos(\alpha_{new}) &\propto \mathbf{w_{new}}^T \mathbf{x} \\ &\propto (\mathbf{w} + \mathbf{x})^T \mathbf{x} \\ &\propto \mathbf{w}^T \mathbf{x} + \mathbf{x}^T \mathbf{x} \\ &\propto cos\alpha + \mathbf{x}^T \mathbf{x} \end{aligned}$$

For $\mathbf{x} \in N$ if $\mathbf{w}.\mathbf{x} \geq 0$ then it means that the angle (α) between this $\mathbf{x}$ and the current $\mathbf{w}$ is less than 90° (but we want α to be greater than 90°)

What happens to the new angle (α_{new}) when $\mathbf{w_{new}} = \mathbf{w} - \mathbf{x}$

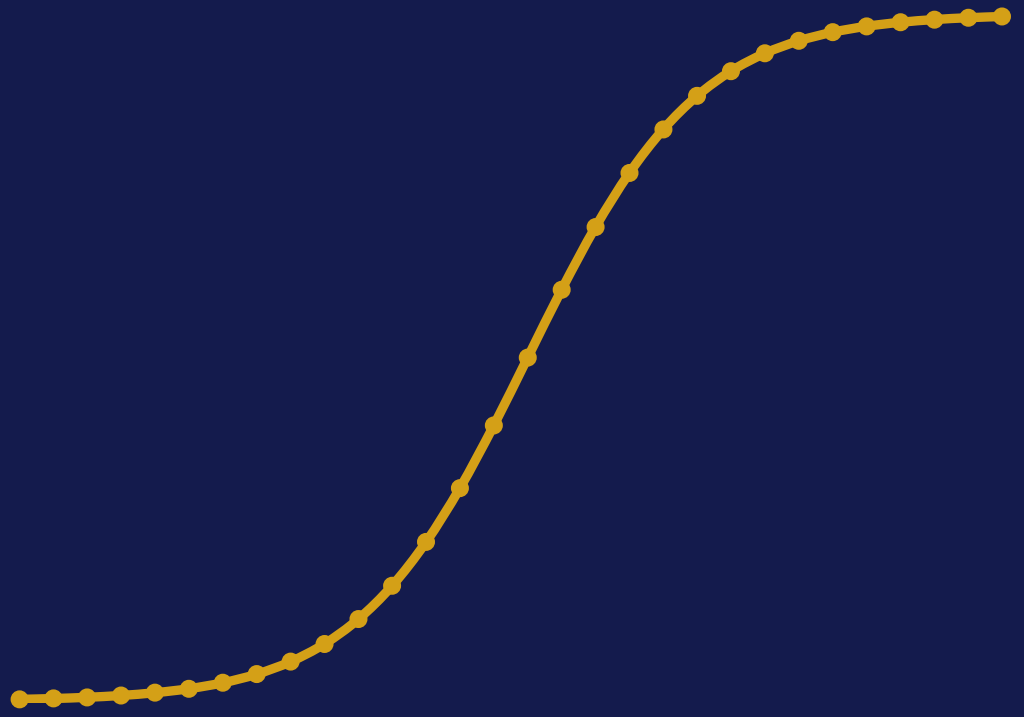
$$
\begin{aligned}
cos(\alpha_{new}) &\propto \mathbf{w_{new}}^T \mathbf{x} \\
&\propto (\mathbf{w} - \mathbf{x})^T \mathbf{x} \\
&\propto \mathbf{w}^T \mathbf{x} - \mathbf{x}^T \mathbf{x} \\
&\propto cos\alpha - \mathbf{x}^T \mathbf{x}
\end{aligned}
$$

DEEP LEARNING · UNIT — FOUNDATIONS

The Sigmoid Neuron

From a hard threshold to a smooth, differentiable decision

Perceptron / MP Neuron → Sigmoid Neuron



Why Move Beyond the Perceptron / MP Neuron?

1

All-or-nothing output

The perceptron's step function fires only 0 or 1 — there is no sense of "how strongly" the neuron believes its output.

2

Abrupt, discontinuous jumps

A tiny change in weights or inputs can flip the output instantly from 0 to 1, making learning unstable.

3

Not differentiable

The step function has a derivative of 0 almost everywhere (undefined at the threshold) — gradient descent has nothing to work with.

4

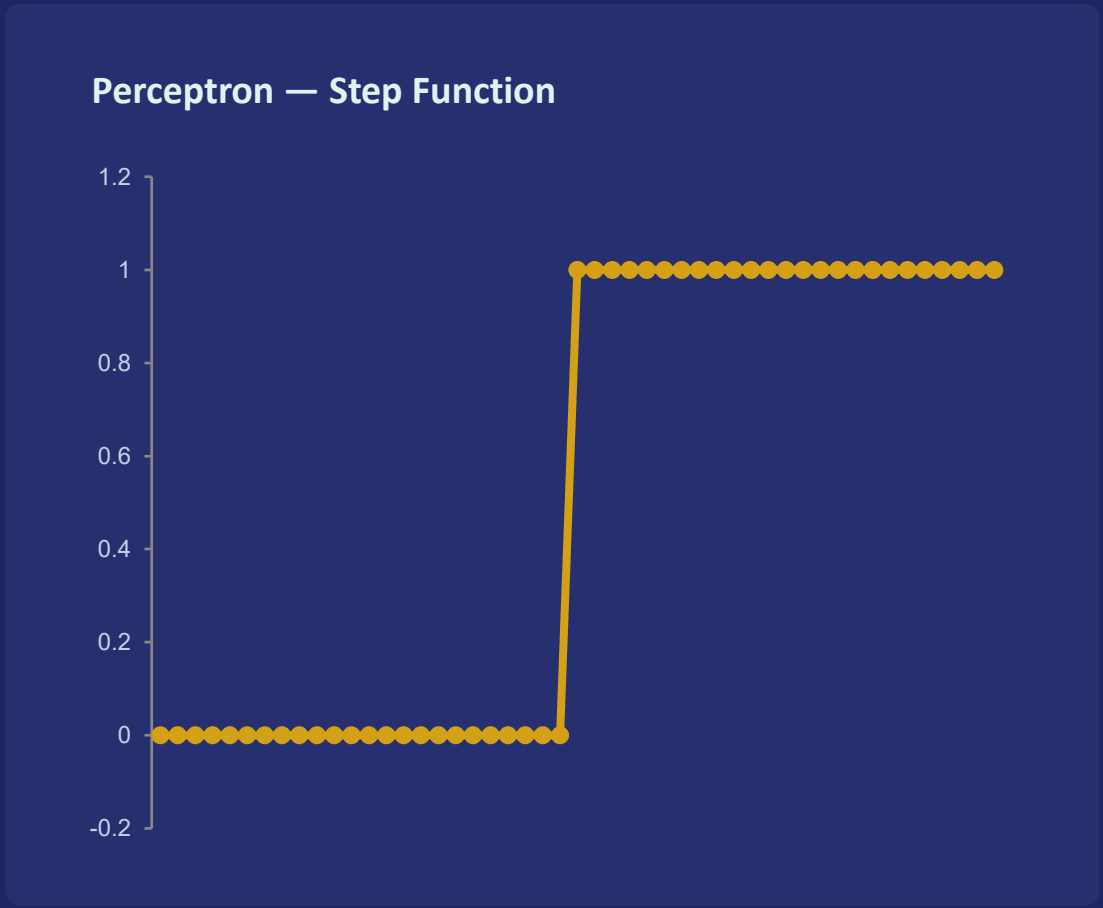
No principled learning rule

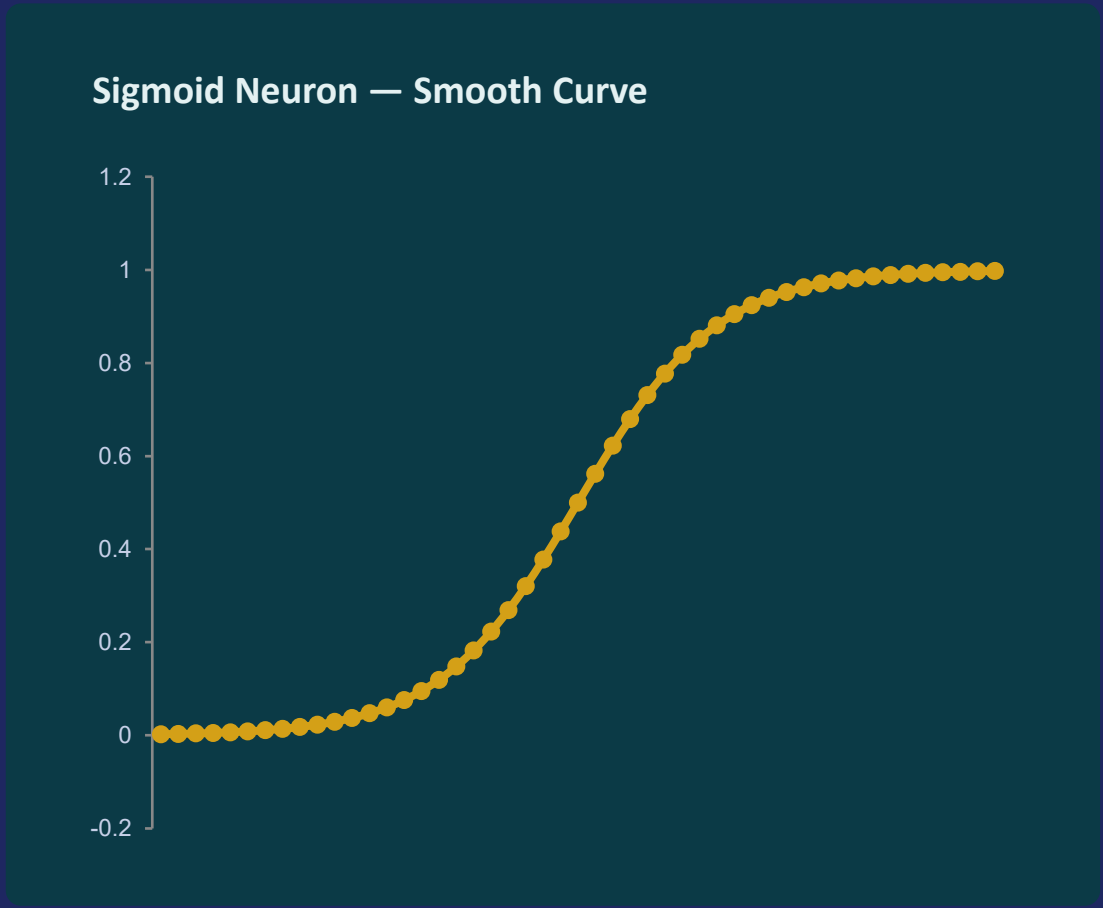
The perceptron update rule is a heuristic; it does not come from minimizing a smooth loss function.

Sigmoid neuron fix: replace the hard step with a smooth, bounded, differentiable curve.

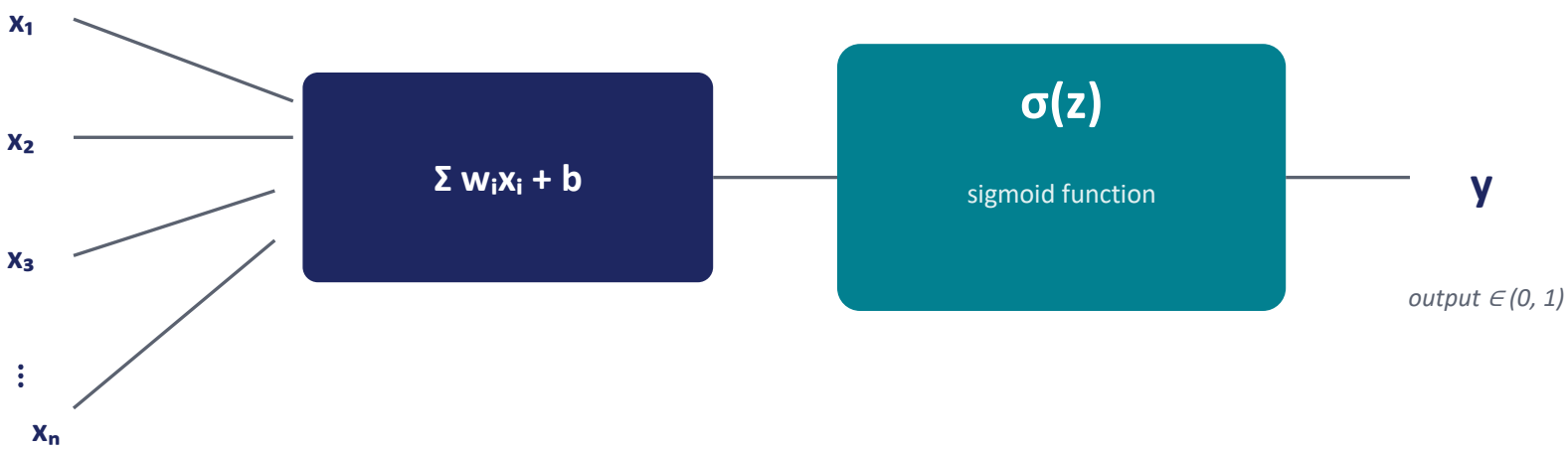
A HARD EDGE VS. A SMOOTH CURVE

Step Function vs. Sigmoid Function





Inputs, Output and Equation



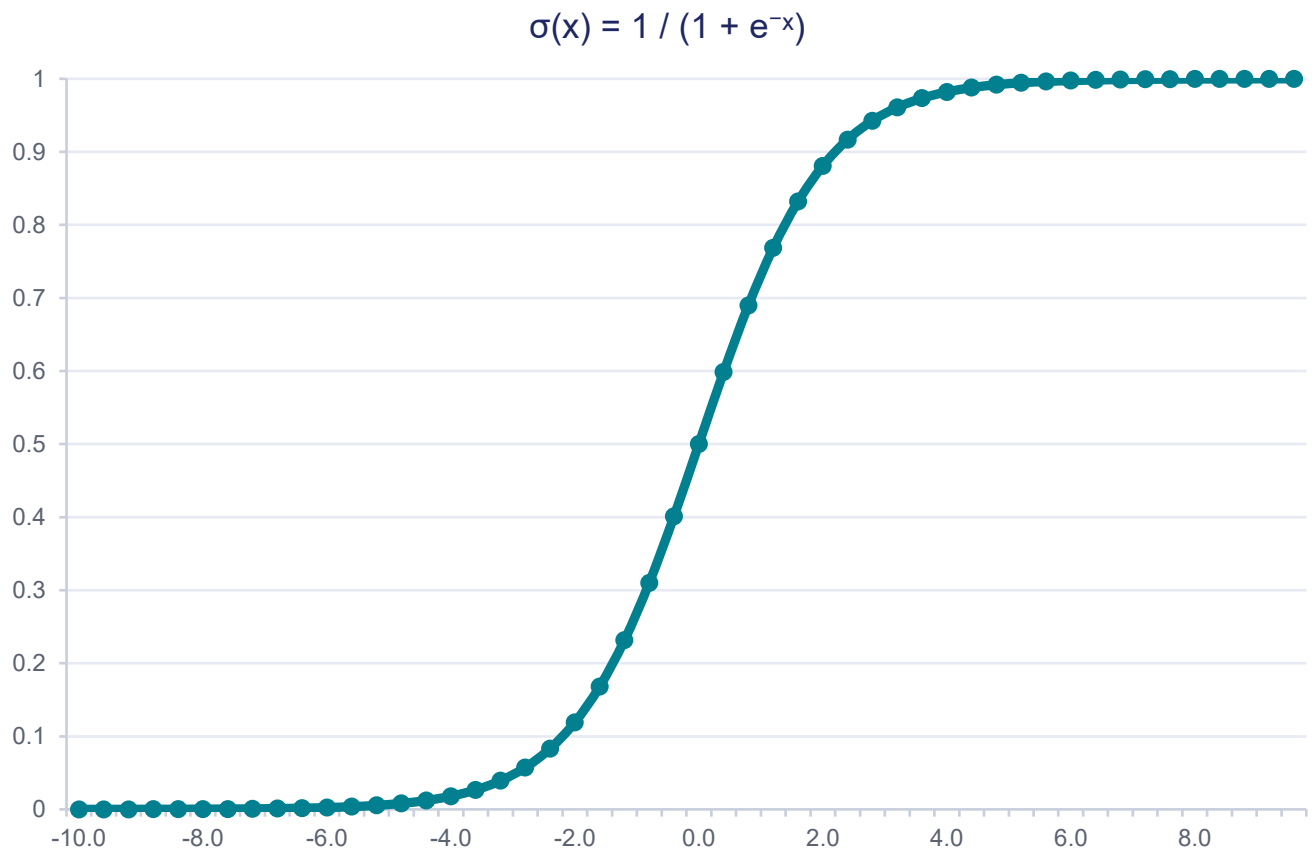
Equation of the Sigmoid Neuron

$$y = \sigma(w \cdot x + b) = 1 / (1 + e^{-(wx+b)})$$

where w = weight vector, x = input vector, b = bias, z = w·x + b (pre-activation)

Unlike the perceptron, inputs and output are real-valued — not restricted to 0/1.

The Sigmoid Function — Graphically

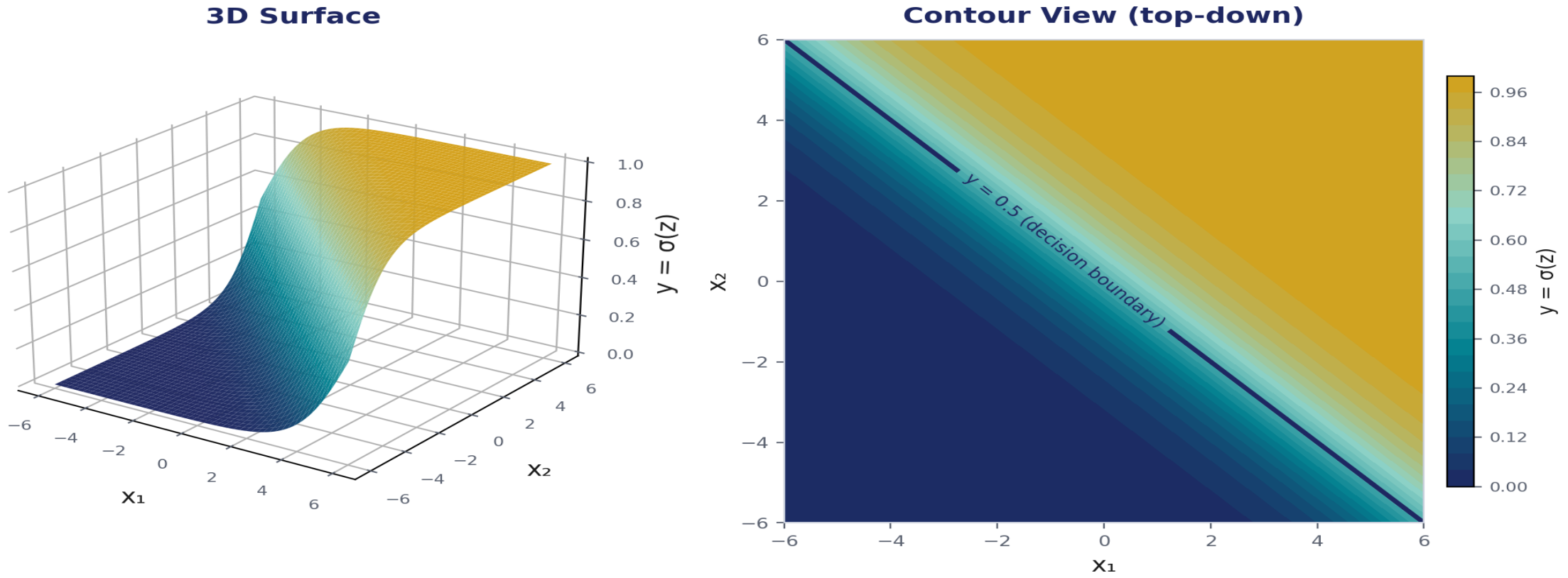


KEY PROPERTIES

- S-shaped curve, bounded between 0 and 1
- Continuous and smooth — no sudden jumps
- Differentiable everywhere → gradients exist
- Steepest around z = 0; flattens at the extremes
- Output can be read as a probability-like confidence

The Sigmoid Function with Two Inputs

With two inputs, $z = w_1x_1 + w_2x_2 + b$ and $y = \sigma(z)$. The 1-D S-curve becomes a smooth S-shaped sheet over the (x_1, x_2) plane.

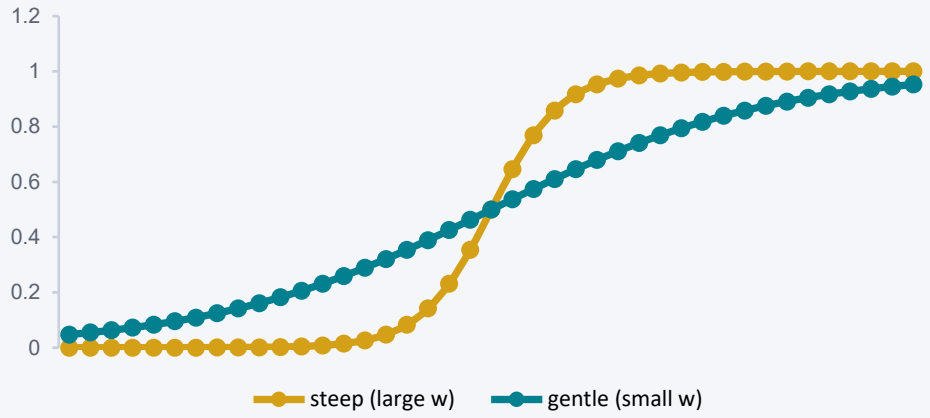


The 0.5 contour (navy line) is a straight decision boundary — sigmoid smooths the sharp perceptron edge into a gradual transition around it.

Parameters of the Sigmoid Neuron

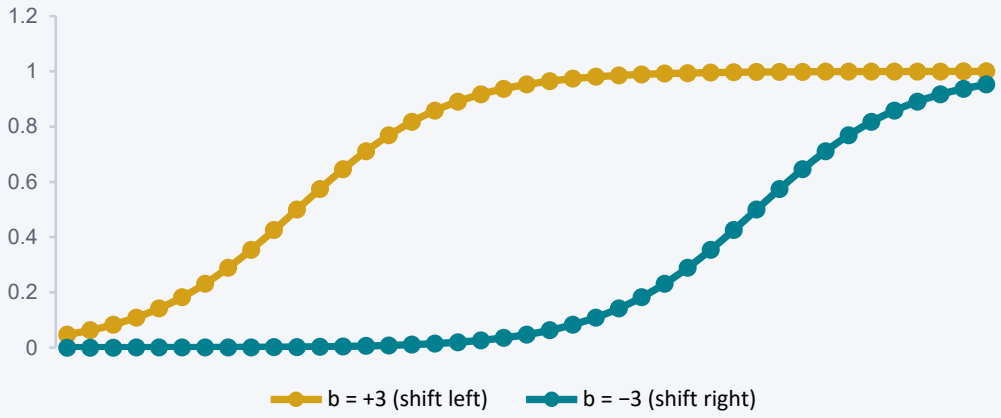
The sigmoid neuron has the same two parameters as the perceptron — **weights (w) and bias (b)** — but they now shape a smooth curve instead of a hard threshold.

w **Weight vector ($w_1, w_2, \ldots, w_n$)**



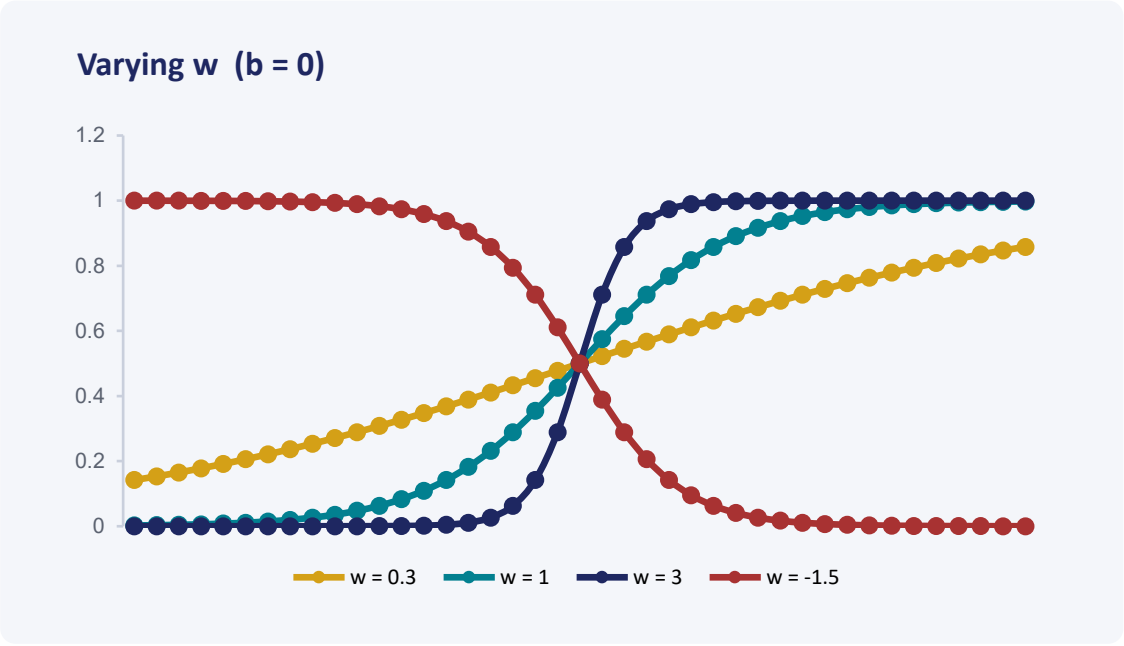
Controls the steepness / slope and orientation of the curve.

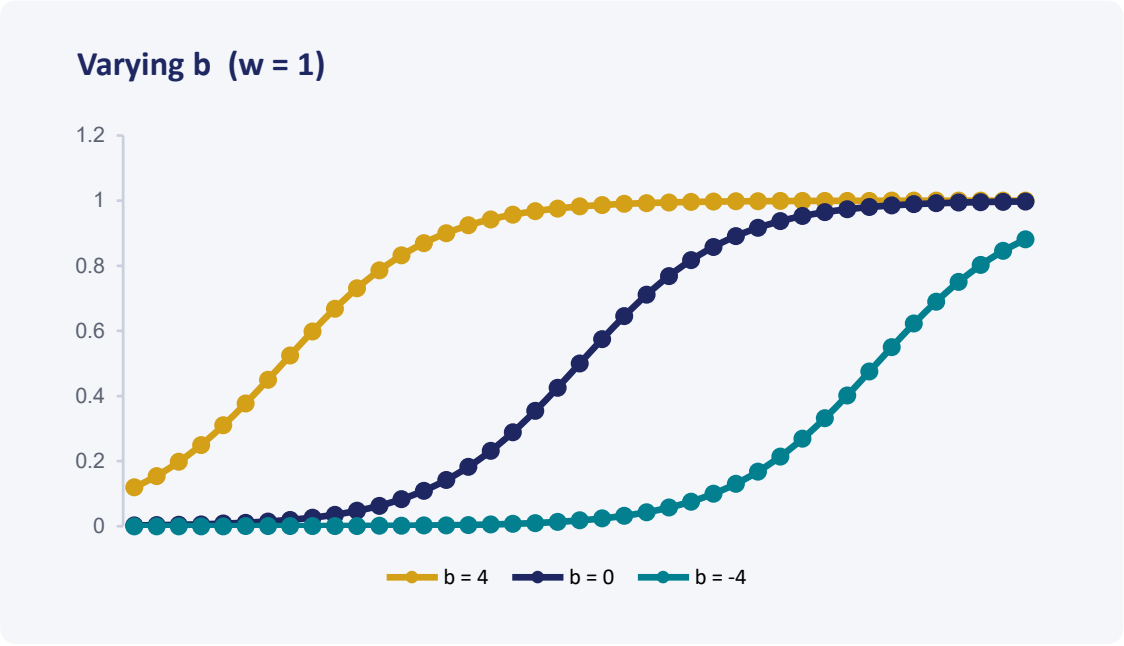
b **Bias (scalar)**



Shifts the curve left or right along the input axis — moves the decision point.

Effect of Changing w and b on the Curve





Change	Effect on the curve
w ↑ (larger magnitude)	Curve becomes steeper — transition from 0 to 1 happens more sharply
w ↓ (smaller magnitude)	Curve becomes gentler / flatter — more gradual transition
w changes sign (+ → −)	Curve flips — becomes a decreasing function of x
b ↑ (more positive)	Curve shifts left — neuron activates for smaller x
b ↓ (more negative)	Curve shifts right — neuron activates for larger x

Learning Algorithm — Gradient Descent

Goal: find w, b that minimize the loss L(w, b) over all training points.

ALGORITHM 1 — Gradient Descent for the Sigmoid Neuron

Input: dataset $\{(x_i, y_i)\}$, learning rate η, epochs T **Output:** learned parameters w, b

```
initialize w, b ← small random values
for epoch = 1 to T do
    for each (xᵢ, yᵢ) in dataset do
        zᵢ ← w·xᵢ + b                          // pre-activation
        fᵢ ← σ(zᵢ)                             // forward pass
    end for
    Compute loss L(w,b) = Σᵢ (yᵢ − fᵢ)²        // compute loss
    Compute partial derivative of loss wrto w - ∂L/∂w   // gradients
    Compute partial derivative of loss wrto b - ∂L/∂b
    w ← w − η·∂L/∂w                            // update w
    b ← b − η·∂L/∂b                            // update b
end for
return w, b
```

Loss Function of the Sigmoid Neuron

Squared Error Loss

$$L(w, b) = \tfrac{1}{2} \Sigma_i \left(y_i - f(x_i) \right)^2 \quad \text{where} \quad f(x_i) = \sigma(w \cdot x_i + b)$$

Sums the squared difference between true label y_i and predicted output $f(x_i)$ across all N examples.

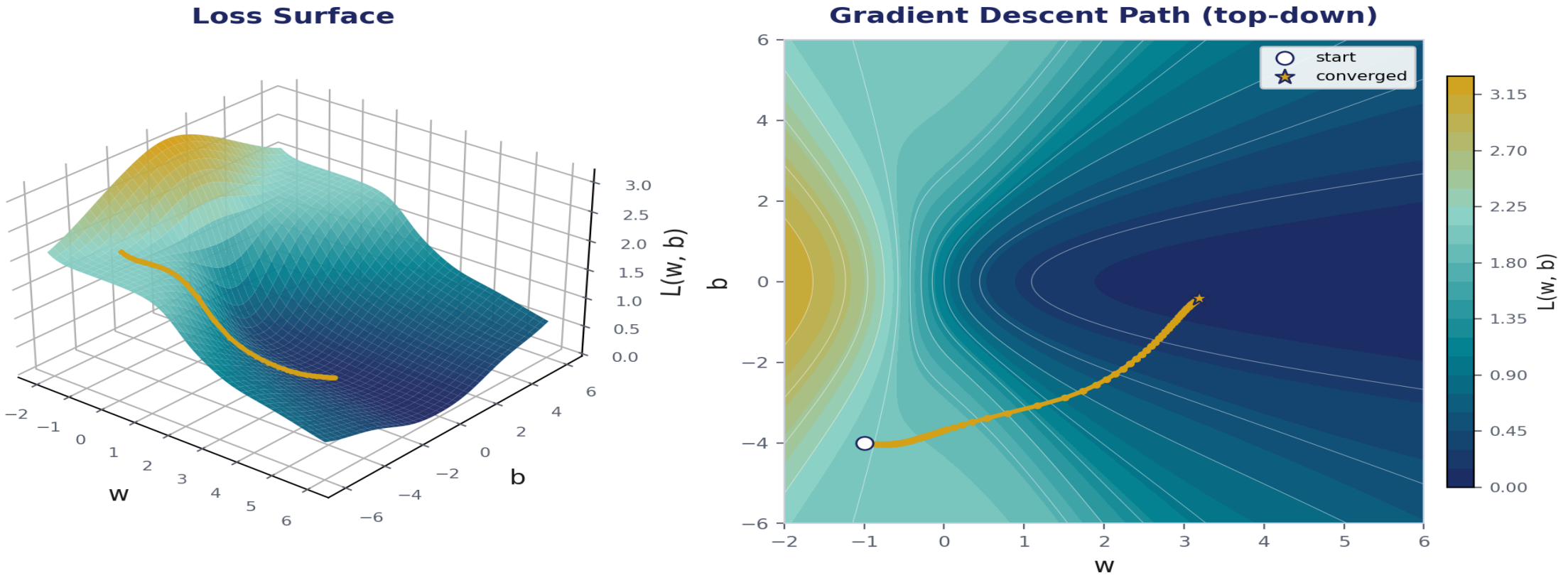
Because σ(z) is smooth and differentiable, L(w, b) is differentiable too — gradients ∂L/∂w and ∂L/∂b can always be computed.

This is exactly what the perceptron's step function could not offer — its loss surface has flat regions with zero gradient almost everywhere.

Minimizing L(w, b) via gradient descent moves the sigmoid curve so its predictions get closer to the true labels.

How the Loss Changes with w and b

L(w, b) forms a smooth bowl-shaped surface over parameter space. Gradient descent walks downhill along this surface toward the minimum loss.



Gold path: each gradient-descent update (w, b) takes a step toward lower loss until it settles near the minimum.

SUMMARY

Perceptron vs. Sigmoid Neuron

Aspect	Perceptron / MP Neuron	Sigmoid Neuron
Activation	Hard step function	Smooth sigmoid, σ(z)
Output range	{0, 1} only	Continuous (0, 1)
Differentiable?	No (undefined at threshold)	Yes, everywhere
Sensitivity to input	Abrupt flip	Smooth, gradual change
Learning rule	Heuristic update rule	Gradient descent on a loss function

Sigmoid neuron: same building blocks as the perceptron (w, x, b) — but a smooth, differentiable activation makes gradient-based learning possible.

How to compute Δw and Δb ?

$\mathcal{L}(\theta) = -[(1 - y) \log(1 - \hat{y}) + y \log \hat{y}]$= Cross Entropy Loss

$\Delta w = \partial \mathcal{L}(\theta)/\partial w = \partial \mathcal{L}(\theta)/\partial \hat{y} \times \partial \hat{y}/\partial w$

$\partial \mathcal{L}(\theta)/\partial \hat{y} = \partial/\partial \hat{y} \{ -(1 - y) \log(1 - \hat{y}) - y \log \hat{y} \}$

$= (-)(-1) \times (1 - y)/(1 - \hat{y}) - y/\hat{y}$

$= [\hat{y}(1 - y) - y(1 - \hat{y})] / [(1 - \hat{y})\hat{y}]$

$= (\hat{y} - y) / [(1 - \hat{y})\hat{y}]$

$$\Delta w = (\hat{y} - y) * x$$

$$\Delta b = (\hat{y} - y) * 1$$

$\partial \hat{y}/\partial w = \partial/\partial w [1 / (1 + e^{-}(wx+b))]$

$= [-1 / (1+e^{-}(wx+b))^2] \times \partial/\partial w (e^{-}(wx+b))$

$= [-1 / (1+e^{-}(wx+b))^2] \times e^{-}(wx+b) \times \partial/\partial w(-(wx+b))$

$= [-1 / (1+e^{-}(wx+b))] \times [e^{-}(wx+b) / (1+e^{-}(wx+b))] \times (-x)$

$= [1 / (1+e^{-}(wx+b))] \times [e^{-}(wx+b) / (1+e^{-}(wx+b))] \times (x)$

$= \hat{y} * (1 - \hat{y}) * x$

```python
def f(w, b, x):
    # sigmoid with parameters w, b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error(w, b):
    err = 0.0
    for x, y in zip(X, Y):
        fx = f(w, b, x)
        err += 0.5*(fx - y)**2
    return err

def grad_b(w, b, x, y):
    fx = f(w, b, x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w, b, x, y):
    fx = f(w, b, x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent():
    w, b, eta = -2, -2, 1.0
    max_epochs = 1000
    for i in range(max_epochs):
        dw, db = 0, 0
        for x, y in zip(X, Y):
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db
```

Learning algorithm

The Four Steps

1. **Model**

   f(w, b, x) — sigmoid neuron prediction

2. **Loss**

   error(w, b) — squared error over all points

3. **Gradients**

   grad_w, grad_b — partial derivatives of the loss

4. **Update**

   w, b nudged opposite the gradient, scaled by η

```
def f(w, b, x):
    # sigmoid with parameters w, b
    return 1.0 / (1.0 + np.exp(-(w*x + b)))

def error(w, b):
    err = 0.0
    for x, y in zip(X, Y):
        fx = f(w, b, x)
        err += 0.5*(fx - y)**2
    return err

def grad_b(w, b, x, y):
    fx = f(w, b, x)
    return (fx - y) * fx * (1 - fx)

def grad_w(w, b, x, y):
    fx = f(w, b, x)
    return (fx - y) * fx * (1 - fx) * x

def do_gradient_descent():
    w, b, eta = -2, -2, 1.0
    max_epochs = 1000
    for i in range(max_epochs):
        dw, db = 0, 0
        for x, y in zip(X, Y):
            dw += grad_w(w, b, x, y)
            db += grad_b(w, b, x, y)
        w = w - eta * dw
        b = b - eta * db
```

Learning Algorithm - Code Changes

```
def error(w, b):
    err = 0.0
    for x, y in zip(X, Y):
        fx = f(w, b, x)
        err += -[(1-y)*math.log(1 - fx , 2)+y*math.log(fx,2)]
    return err
```

```
def grad_b(w, b, x, y):
    fx = f(w, b, x)
    return (fx - y)
```

```
def grad_w(w, b, x, y):
    fx = f(w, b, x)
    return (fx - y) * x
```

$$\Delta w = (\hat{y} - y) * x$$

Interactive Quiz



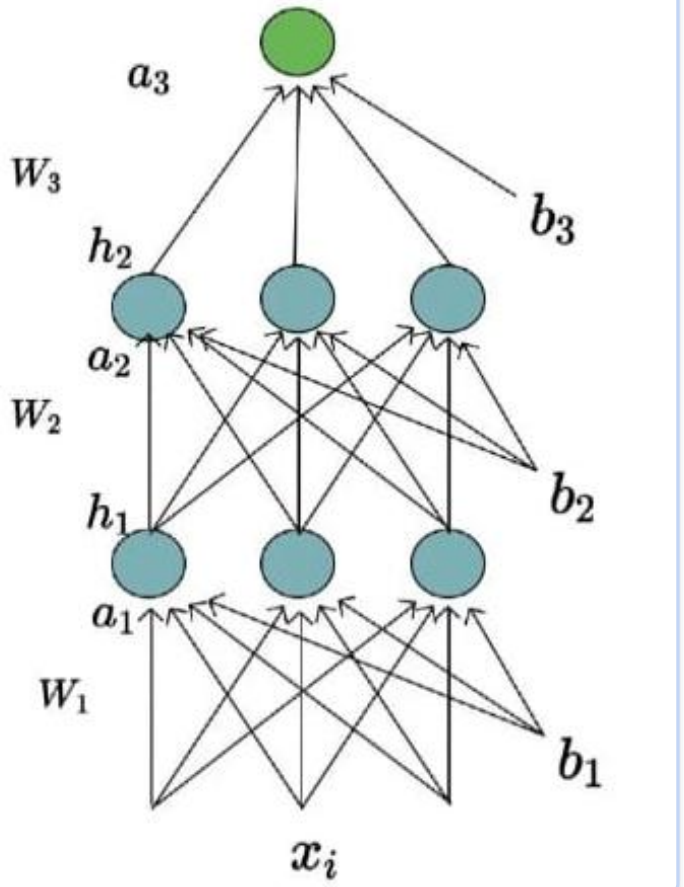
DEEP LEARNING · FOUNDATIONS

Generalizing from a single neuron to a deep, multi-layer network: matrix notation, the gradient-descent update rule, the full derivative matrix, and the chain-rule calculus that makes it all work.

Model Notation | Learning Algorithm | Derivatives Needed | Calculus Recap

Can we clarify the terminology a bit?

Model — From Scalars to Matrix Notation



Deep network: 100 inputs → 10 neurons → 3 neurons → 2 outputs

Matrix Notation

Weight matrix W1 (10 × 100) — one row per hidden-1 neuron, one column per input:

```
W1 = [ w111   w112 ... w1199   w11100 ]
     [ w121   w122 ... w1299   w12100 ]
     [   .      .        .        .  ]
     [ w1,10,1 ... w1,10,99  w1,10,100 ]
```

Input vector X (100 × 1):

X = [x1, x2, … , x100]^T

Reading the subscripts: w1,j,i = weight in layer 1, feeding neuron j, from input i.

Pre-Activation Calculations

Every neuron j in layer 1 does a weighted sum of all 100 inputs, plus its bias:

a11 = w111·x1 + w112·x2 + w113·x3

+ … + w11100·x100 + b11

a12 = w121·x1 + w122·x2 + w123·x3

+ … + w12100·x100 + b12

⋮

a1,10 = w1,10,1·x1 + w1,10,2·x2

+ … + w1,10,100·x100 + b1,10

In vector form: a1 = W1·X + b1

Can we use the same Gradient Descent algorithm as before?

Learning Algorithm — Same Recipe, More Parameters

The Algorithm

Initialise w, b

Iterate over data:

compute ŷ

compute L(w, b)

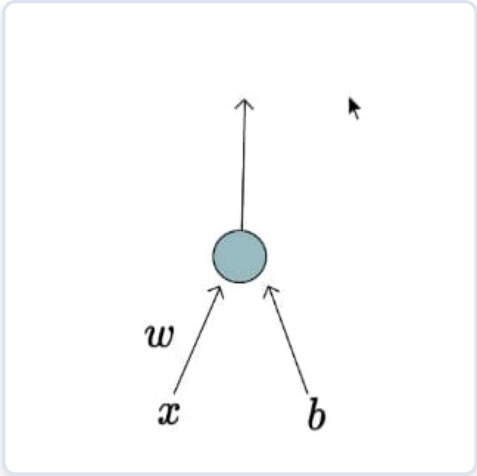
w111 = w111 - η·Δw111

w112 = w112 - η·Δw112

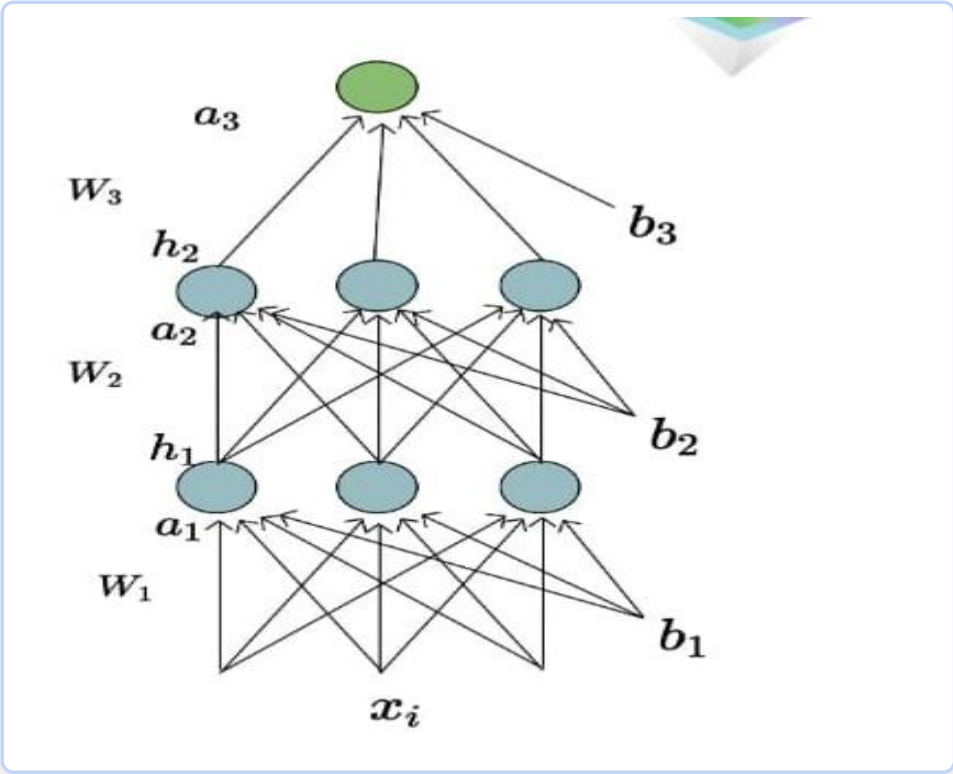
....

w313 = w313 - η·Δw313

till satisfied



Earlier: single neuron (w, b)



What changes

Earlier: w, b

Now: w111, w112, ...

What stays the same

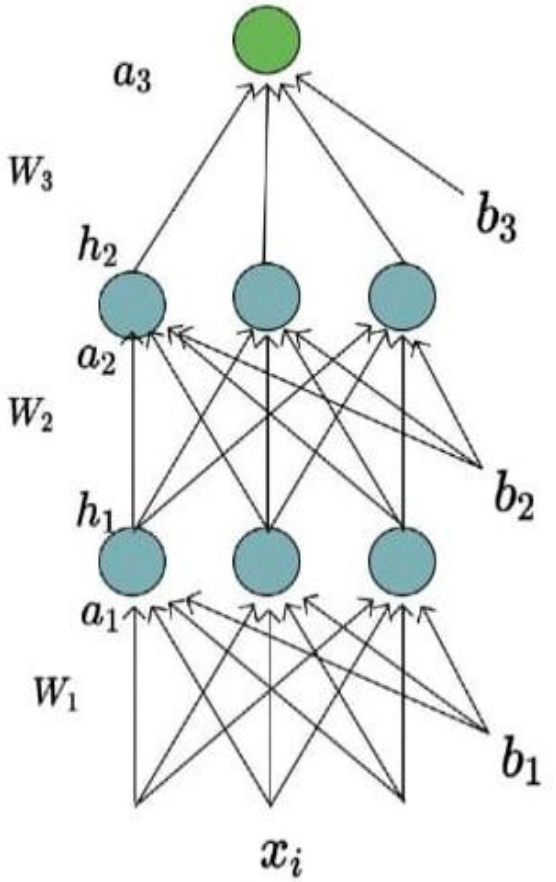
Earlier: L(w, b)

Now: L(w111, w112, ... , b_Lk)

Same update rule — just applied to many more parameters.

How many derivatives do we need to compute, and how do we compute them?

The Full Derivative Matrix



Network with L layers, k neurons per layer

Every Weight and Bias Needs Its Own Gradient

For a network with L layers, we need one partial derivative of the loss for every single weight and bias, arranged as a big gradient matrix ∂L(θ)/∂θ:

$$\begin{matrix} \partial L/\partial w_{111} & \dots & \partial L/\partial w_{11n} & \partial L/\partial w_{211} & \dots & \partial L/\partial w_{21n} & \dots & \partial L/\partial w_{L,11} & \dots & \partial L/\partial w_{L,1k} & \partial L/\partial b_{11} & \dots & \partial L/\partial b_{L1} \\ \partial L/\partial w_{121} & \dots & \partial L/\partial w_{12n} & \partial L/\partial w_{221} & \dots & \partial L/\partial w_{22n} & \dots & \partial L/\partial w_{L,21} & \dots & \partial L/\partial w_{L,2k} & \partial L/\partial b_{12} & \dots & \partial L/\partial b_{L2} \\ & & \vdots & & & \vdots & & & & \vdots & & & \\ \vdots & & & & & & & & & & & & \\ \partial L/\partial w_{1n1} & \dots & \partial L/\partial w_{1nn} & \partial L/\partial w_{2n1} & \dots & \partial L/\partial w_{2nn} & \dots & \partial L/\partial w_{L,n1} & \dots & \partial L/\partial w_{L,nk} & \partial L/\partial b_{1n} & \dots & \partial L/\partial b_{Lk} \end{matrix}$$

This is exactly what backpropagation computes efficiently — one backward sweep through the network gives every entry of this matrix using the chain rule.

Can we do a quick recap of some basic calculus?

(Partial) Derivatives, Gradients

Basic Derivative Rules

$$\frac{d(e^x)}{dx} = e^x$$

$$\frac{d(x^2)}{dx} = 2x$$

$$\frac{d(1/x)}{dx} = -1/x^2$$

These three rules are all we need — every gradient in the network is built by chaining them together.

Chain Rule — Nested Function

Differentiate $e^{(x^2)}$ with respect to x:

$$\frac{d(e^{x^2})}{dx} = \frac{d(e^{x^2})}{d(x^2)} \cdot \frac{d(x^2)}{dx}$$

$$= \frac{de^z}{dz} \cdot \frac{dx^2}{dx} \quad [\text{let } z = x^2]$$

$$= (e^z)\cdot(2x)$$

$$= (e^{x^2})\cdot(2x)$$

$$= 2x\cdot e^{x^2}$$

Chain Rule — Doubly Nested

Differentiate $e^{(e^{x^2})}$ w.r.t. x:

$$= \frac{d(e^{e^{x^2}})}{d(e^{x^2})} \cdot \frac{d(e^{x^2})}{dx}$$

$$= (e^z)\cdot(2x\cdot e^{x^2}) \quad [z = e^{x^2}]$$

$$= 2x\cdot e^{x^2}\cdot e^{(e^{x^2})}$$

Differentiate $1/e^{(x^2)}$ w.r.t. x:

$$= (-1/z^2)\cdot(2x\cdot e^{x^2}) \quad [z = e^{x^2}]$$

$$= -2x\cdot e^{x^2} / (e^{x^2})^2$$

NEURAL NETWORK · WORKED EXAMPLE

Learning Algorithm

Forward propagation, loss, and backpropagation —
worked step by step with real numbers.

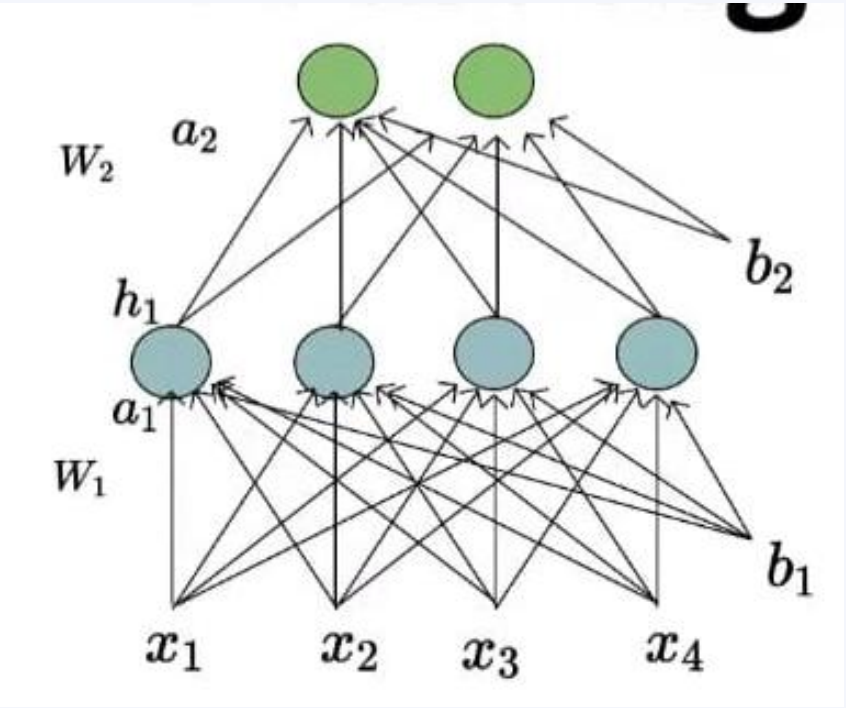
Network: 4 inputs → 4 hidden (sigmoid) → 2 outputs (softmax)

$x = [2 \;\; 5 \;\; 3 \;\; 3] \qquad y = [1 \;\; 0]$

FORWARD PASS

Computing the Network Output

Given inputs x and weights W_1, W_2 — propagate forward to ŷ



$b = [\ 0\ \ 0\ ]$

$W_1 = [\ 0.1\ \ 0.3\ \ 0.8\ \ -0.4\ ]$

W_1 = 4×4 (rows 2-4 continue similarly) W_2 = 2×4

$x = [2\ 5\ 3\ 3]$ $y = [1\ 0]$

OUTPUT

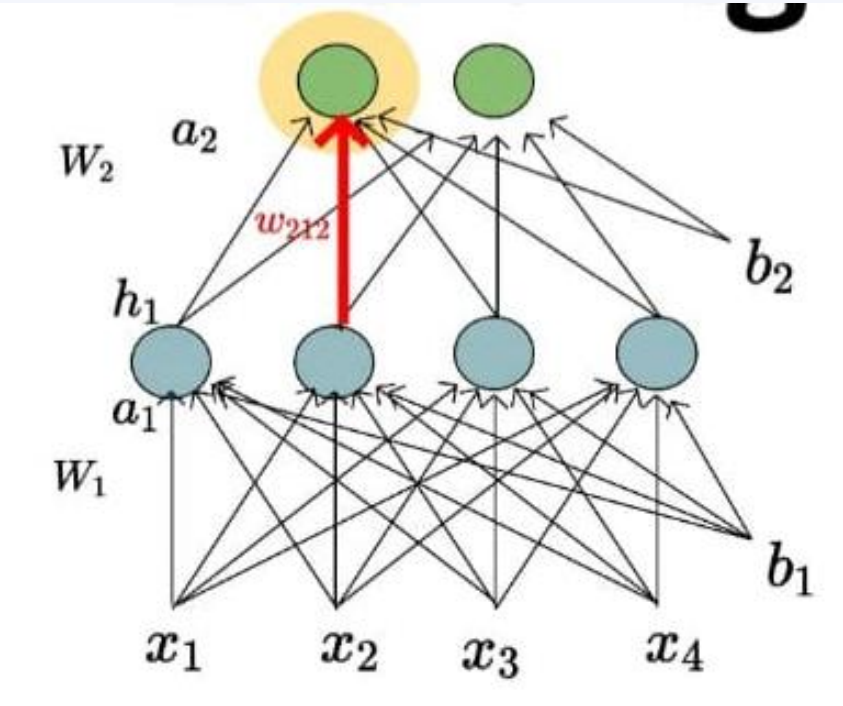
$a_1 = W_1 \cdot x + b_1 =$	`[ 2.9  1.4  2.1  2.3 ]`
$h_1 = sigmoid(a_1) =$	`[ 0.95  0.80  0.89  0.91 ]`
$a_2 = W_2 \cdot h_1 + b_2 =$	`[ 1.66  0.45 ]`
$\hat{y} = softmax(a_2) =$	`[ 0.77  0.23 ]`

Prediction check: $\hat{y}_1 = 0.77$ is the larger softmax value, matching the true label y = [1, 0]. The network's forward pass already leans toward the correct class — backprop below refines exactly how much each weight contributed.

BACKWARD PASS · OUTPUT LAYER

Gradient for Weight w_{212}

A single-hop chain rule: this weight feeds directly into output neuron 1



w_{212} connects hidden neuron h_{12} → output a_{21}

(highlighted in red on the diagram above)

CHAIN RULE

$$\partial L / \partial w_{212} = (\partial L/\partial a_{21}) \cdot (\partial a_{21}/\partial w_{212})$$

$$= (\partial L/\partial \hat{y}_1) \cdot (\partial \hat{y}_1/\partial a_{21}) \cdot (\partial a_{21}/\partial w_{212})$$

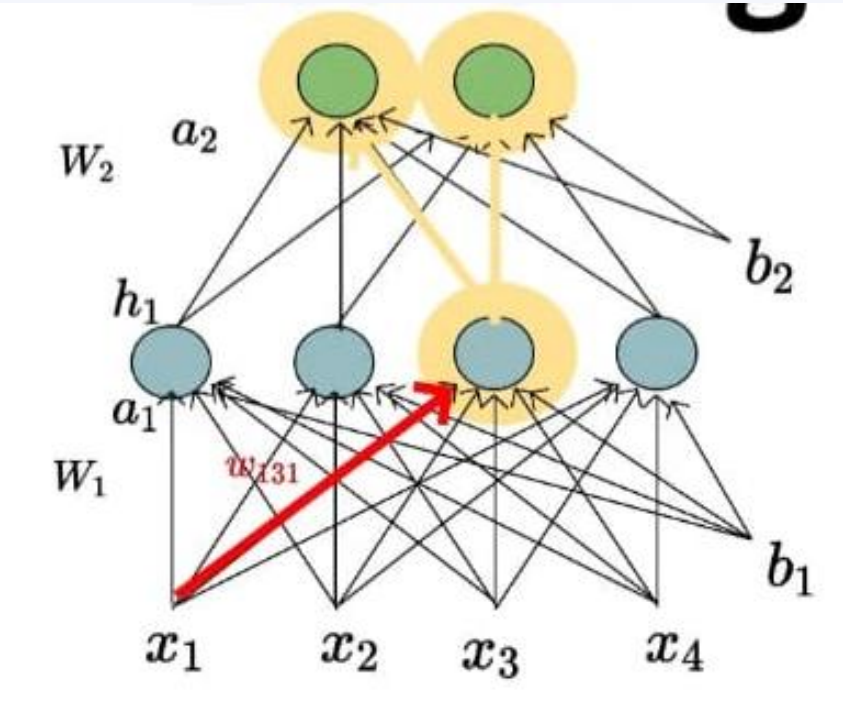
$\partial L / \partial \hat{y}_1$	$= -2(y_1 - \hat{y}_1)$	`= -2(1 - 0.77) = -0.46`
$\partial \hat{y}_1 / \partial a_{21}$	$= \hat{y}_1(1 - \hat{y}_1)(-a_{22})$	`= 0.77(0.23)(-0.45) = -0.0797`
$\partial a_{21} / \partial w_{212}$	$= h_{12}$	`= 0.80`

$$\partial L / \partial w_{212} = (-0.46)(-0.0797)(0.80) = \mathbf{0.0293}$$

BACKWARD PASS · HIDDEN LAYER

Gradient for Weight w_{131}

A two-path chain rule: this weight influences both output neurons through h_{13}



w_{131} connects input x_1 → hidden h_{13} → both outputs

h_{13} feeds both a_{21} and a_{22} — gradients from both paths must be summed

CHAIN RULE — TWO PATHS SUMMED

$$\frac{\partial L}{\partial w_{131}} = \left[\left(\frac{\partial L}{\partial \hat{y}_1}\cdot\frac{\partial \hat{y}_1}{\partial a_{21}}\cdot\frac{\partial a_{21}}{\partial h_{13}}\right) + \left(\frac{\partial L}{\partial \hat{y}_2}\cdot\frac{\partial \hat{y}_2}{\partial a_{22}}\cdot\frac{\partial a_{22}}{\partial h_{13}}\right) \right] \cdot \left(\frac{\partial h_{13}}{\partial a_{13}}\right) \cdot \left(\frac{\partial a_{13}}{\partial w_{131}}\right)$$

$\frac{\partial L}{\partial \hat{y}_1} = -2(y_1-\hat{y}_1)$	**-0.46**
$\frac{\partial L}{\partial \hat{y}_2} = -2(y_2-\hat{y}_2)$	**0.46**
$\frac{\partial \hat{y}_1}{\partial a_{21}} = \hat{y}_1(1-\hat{y}_1)(-a_{22})$	**-0.0797**
$\frac{\partial \hat{y}_2}{\partial a_{22}} = \hat{y}_2(1-\hat{y}_2)(-a_{21})$	**-0.2940**
$\frac{\partial a_{21}}{\partial h_{13}} = w_{213}$	**0.20**
$\frac{\partial a_{22}}{\partial h_{13}} = w_{223}$	**0.30**
$\frac{\partial h_{13}}{\partial a_{13}} = h_{13}(1-h_{13})$	**0.0979**
$\frac{\partial a_{13}}{\partial w_{131}} = x_1$	**2**

$$\frac{\partial L}{\partial w_{131}} = [(-0.46)(-0.0797)(0.2) + (0.46)(-0.2940)(0.3)] \times 0.0979 \times 2$$

$$= \mathbf{-0.00651}$$

SUMMARY

Backpropagation is repeated application of the chain rule: trace every path from the loss back to a weight, multiply local derivatives along each path, and sum paths that share an edge.

1. Forward Pass

Compute a_1, h_1, a_2, $\hat{y}$ layer by layer.

2. Loss Gradient

$\partial L/\partial \hat{y}$ seeds every backward path.

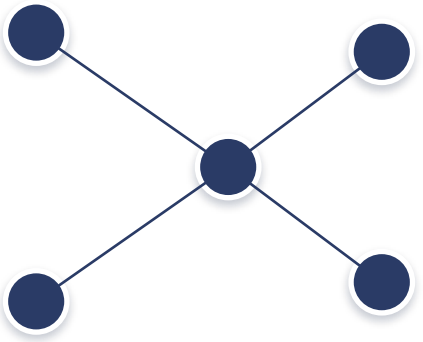
3. Single-Path Weights

Multiply local derivatives once (e.g. w_{212}).

4. Shared-Path Weights

Sum contributions from every path (e.g. w_{131}).

DEEP LEARNING · NEURAL NETWORKS



A 2-Hidden-Layer Network for Binary Classification — Architecture, Forward Pass, Backpropagation & Gradient-Descent Updates

Architecture | Forward Pass | Backpropagation | Weight Updates

Prepared for a technical training session

1. Network Setup: 2 Hidden Layers, Binary Classification

Every calculation in this deck traces through this exact configuration

Architecture

Input layer

2 features: x1 , x2

Hidden layer 1

2 neurons | weights W1 (2×2) | bias b1 (2×1)

Activation: ReLU(z) = max(0, z)

Hidden layer 2

2 neurons | weights W2 (2×2) | bias b2 (2×1)

Activation: ReLU(z) = max(0, z)

Output layer

1 neuron | weights W3 (2×1) | bias b3 (scalar)

Activation: Sigmoid(z) = 1 / (1 + e^-z) → $\hat{y} \in (0,1)$

Loss function

Binary Cross-Entropy: $L = -[\, y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y}) \,]$

Worked Example Used Throughout This Deck

Input: x1 = 0.50 x2 = 0.80 Target label: y = 1

W1 = [0.15 0.20] b1 = [0.10, 0.10]
[0.25 0.30]

W2 = [0.40 0.50] b2 = [0.20, 0.20]
[0.45 0.55]

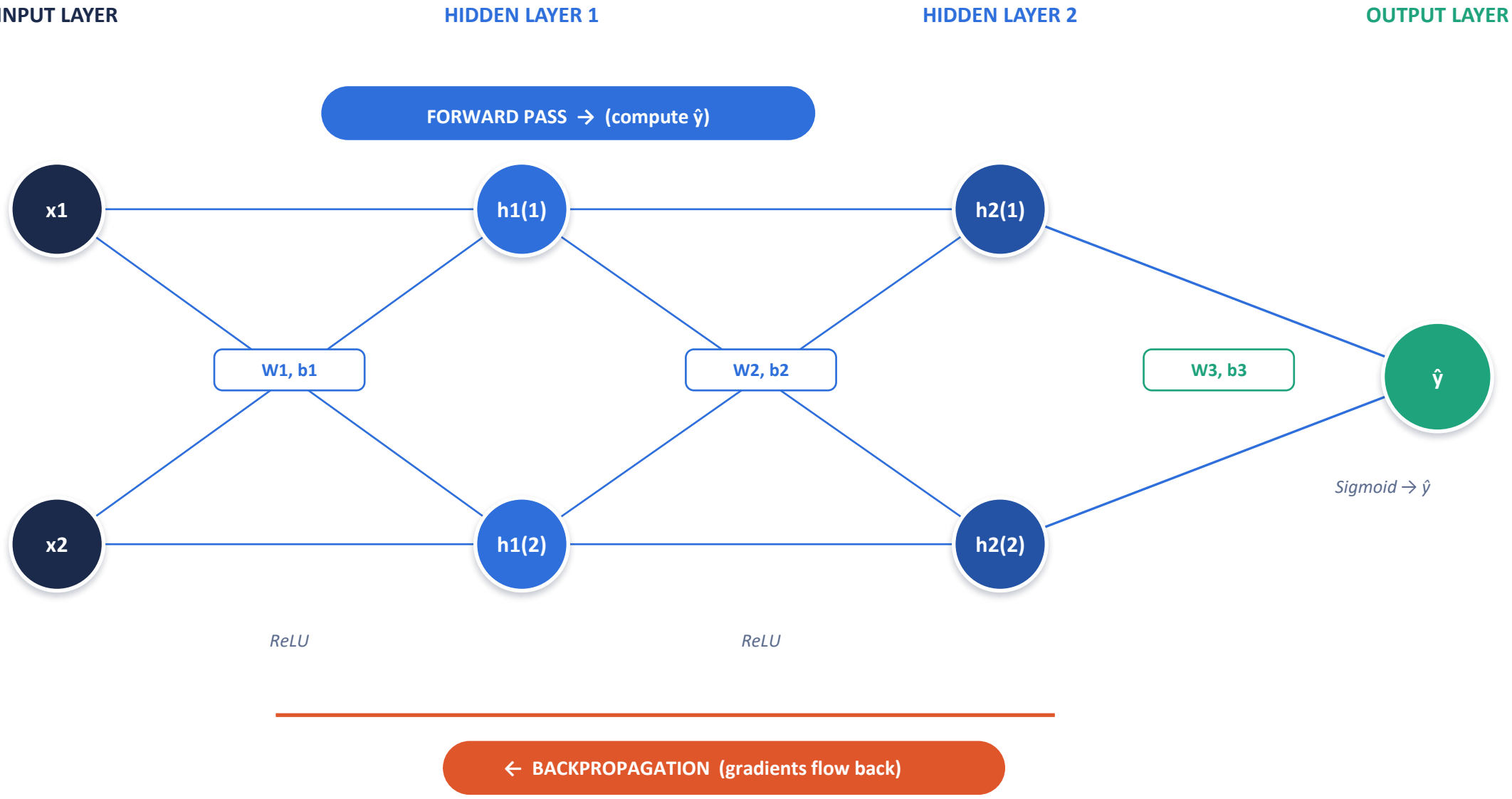
W3 = [0.60 , 0.65] b3 = 0.10

Learning rate: η = 0.1

We will pass this input all the way forward to a prediction $\hat{y}$, compute the loss, then propagate the error all the way back to get every gradient — using these exact numbers on the following slides.

2. Network Diagram — Connections, Weights, Bias & Output

Blue arrows trace the forward pass; the orange path (bottom) shows how gradients flow backward during training



3. Forward Pass — Hidden Layer 1

General rule, then substituted with the worked example

General Formula

For every neuron j in the layer:

$$z1(j) = \Sigma\ xi \cdot w1(i,j)\ +\ b1(j)$$

$$a1(j) = ReLU(\ z1(j)\ ) = max(0, z1(j))$$

In matrix form:

$$z1 = W1^{T}\ x + b1$$

$$a1 = ReLU(z1)$$

z1, a1 ∈ $\mathbb{R}^2$ (one value per hidden-1 neuron)

Substituting the Example

Neuron h1(1):

z1(1) = 0.5(0.15) + 0.8(0.25) + 0.10 = 0.375

a1(1) = ReLU(0.375) = 0.375

Neuron h1(2):

z1(2) = 0.5(0.20) + 0.8(0.30) + 0.10 = 0.440

a1(2) = ReLU(0.440) = 0.440

Output of Hidden Layer 1:

a1 = [0.375, 0.440]

This vector becomes the input to Hidden Layer 2.

3. Forward Pass — Hidden Layer 2, Output & Loss

Continuing the same forward computation to a final prediction

Hidden Layer 2

$z2 = W2^T a1 + b2$

$z2(1) = 0.375(0.4)+0.44(0.45)+0.2$

$= 0.548$

$z2(2) = 0.375(0.5)+0.44(0.55)+0.2$

$= 0.630$

$a2 = ReLU(z2) = [0.548, 0.630]$

Output Layer

$z3 = W3^T a2 + b3$

$z3 = 0.548(0.6)+0.63(0.65)+0.1$

$= 0.838$

$\hat{y} = \sigma(z3) = 1 / (1+e^{-0.838})$

$\hat{y} \approx 0.698$

Predicted class: 1 (since $\hat{y} > 0.5$)

Loss

Binary Cross-Entropy:

$L = -[y \cdot \log \hat{y} + (1-y) \cdot \log(1-\hat{y})]$

With $y = 1$:

$L = -\log(0.698)$

$L \approx 0.359$

This loss value is where backpropagation begins.

4. Backpropagation — Output Layer Gradients

Chain rule: gradient of loss flows backward, layer by layer

Chain Rule at the Output

The key simplification: for sigmoid + BCE together,

$$\partial L/\partial z3 = \hat{y} - y$$

Then, for the weights and bias feeding the output:

$$\partial L/\partial W3 = \partial L/\partial z3 \cdot a2^T$$

$$\partial L/\partial b3 = \partial L/\partial z3$$

And to keep propagating backward into Hidden Layer 2:

$$\partial L/\partial a2 = W3 \cdot \partial L/\partial z3$$

Substituting the Example

$\partial L/\partial z3 = 0.698 - 1 = -0.302$

$\partial L/\partial w3(1) = -0.302 \times 0.548 = -0.1655$

$\partial L/\partial w3(2) = -0.302 \times 0.630 = -0.1901$

$\partial L/\partial b3 = -0.302$

Gradient passed to Hidden Layer 2:

$\partial L/\partial a2(1) = -0.302 \times 0.60 = -0.1812$

$\partial L/\partial a2(2) = -0.302 \times 0.65 = -0.1963$

4. Backpropagation — Hidden Layer 2 Gradients

The gradient passes through the ReLU derivative before reaching W2

Chain Rule at Hidden Layer 2

ReLU derivative: ReLU'(z) = 1 if z>0, else 0

$$\partial L/\partial z2 = \partial L/\partial a2 \odot \text{ReLU}'(z2)$$

$$\partial L/\partial W2 = \partial L/\partial z2 \cdot a1^{T}$$

$$\partial L/\partial b2 = \partial L/\partial z2$$

Propagate further back into Hidden Layer 1:

$$\partial L/\partial a1 = W2 \cdot \partial L/\partial z2$$

Substituting the Example

z2(1)=0.548>0 and z2(2)=0.63>0 → ReLU'=1 for both

$\partial L/\partial z2(1) = -0.1812$, $\partial L/\partial z2(2) = -0.1963$

$\partial L/\partial w2(11) = -0.1812 \times 0.375 = -0.0680$

$\partial L/\partial w2(12) = -0.1963 \times 0.375 = -0.0736$

$\partial L/\partial w2(21) = -0.1812 \times 0.440 = -0.0797$

$\partial L/\partial w2(22) = -0.1963 \times 0.440 = -0.0864$

$\partial L/\partial b2 = [-0.1812, -0.1963]$

$\partial L/\partial a1(1) = -0.1812(0.4) + -0.1963(0.5) = -0.1706$

$\partial L/\partial a1(2) = -0.1812(0.45) + -0.1963(0.55) = -0.1895$

4. Backpropagation — Hidden Layer 1 Gradients

The final leg: gradients reach the weights closest to the input

Chain Rule at Hidden Layer 1

$$\partial L/\partial z1 = \partial L/\partial a1 \odot \text{ReLU}'(z1)$$

$$\partial L/\partial W1 = \partial L/\partial z1 \cdot x^T$$

$$\partial L/\partial b1 = \partial L/\partial z1$$

This completes one full backward pass — every weight and bias in the network now has a gradient telling us which direction reduces the loss.

Substituting the Example

$z1(1)=0.375>0$, $z1(2)=0.44>0$ → ReLU'=1 for both

$\partial L/\partial z1(1) = -0.1706$, $\partial L/\partial z1(2) = -0.1895$

$\partial L/\partial w1(11) = -0.1706 \times 0.5 = -0.0853$

$\partial L/\partial w1(12) = -0.1895 \times 0.5 = -0.0948$

$\partial L/\partial w1(21) = -0.1706 \times 0.8 = -0.1365$

$\partial L/\partial w1(22) = -0.1895 \times 0.8 = -0.1516$

$\partial L/\partial b1 = [-0.1706, -0.1895]$

5. Training the Network — Gradient Descent Updates

Every weight and bias moves a small step against its gradient, then the cycle repeats

Update Rule

For every parameter θ (each weight or bias):

$$\theta_{new} = \theta_{old} - \eta \cdot \partial L/\partial \theta$$

η (eta) = learning rate = 0.1 in our example

The gradient points toward increasing loss, so we step in the opposite direction.

Applying It to W3 (output layer)

w3(1)_new = 0.60 - 0.1×(-0.1655) = 0.6165
w3(2)_new = 0.65 - 0.1×(-0.1901) = 0.6690
b3_new = 0.10 - 0.1×(-0.302) = 0.1302

The Training Loop (per epoch)

1. Forward pass — compute ŷ for a batch of inputs
2. Compute loss L (e.g., binary cross-entropy)
3. Backpropagation — compute ∂L/∂θ for every weight and bias via the chain rule
4. Update every parameter: θ ← θ - η·∂L/∂θ
5. Repeat for all batches, across many epochs, until the loss converges

Result: W1, b1, W2, b2, W3, b3 are gradually tuned so ŷ moves closer to the true label y.

Architecture	2 inputs → Hidden(2, ReLU) → Hidden(2, ReLU) → Output(1, Sigmoid) for binary classification.
Forward Pass	$z = W^T a + b$ at every layer, then an activation — repeated forward until $\hat{y}$ is produced.
Loss	Binary cross-entropy measures how far $\hat{y}$ is from the true label y.
Backpropagation	The chain rule pushes $\partial L/\partial z3 = \hat{y} - y$ backward, layer by layer, to every weight and bias.
Training	Gradient descent repeats forward → loss → backward → update until the network converges.